DD: PLM-CFG-05 — Adjustment Type Catalog

Developer implementation guide: `DD_PLM-CFG-05-Adjustment_Type_Catalog-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_PLM-CFG-05-Adjustment_Type_Catalog-v1.0.md`.

Verdict: ■ New | **Wave:** 2 | **Group P:** Catalogs (PLM family) | **Version:** v1.0 | **Module:** Product Lifecycle Management — Adjustment Type Catalog The operator-extensible catalog of **adjustment reason codes** + their associated business rules: what kind of adjustment is this (credit / debit / refund / waiver), what approval workflow applies, which GL account it posts to, what scope is permitted (full invoice / line-item / free-amount), and what direction is allowed (only credit, only debit, or both). Owned at operator scope. Referenced by BIL-02-ADJ-01 Invoice Adjustments (every adjustment carries a reason code from this catalog), by BIL-05 Wallet Refunds (refund-from-wallet operations), by OSR-01 Stock Chain (deposit refunds at disconnect), and by BIL-04 Dunning Engine (dunning waivers and write-offs). This is **catalog-only** — PLM-CFG-05 does not propose, approve, or post adjustments. The adjustment lifecycle is owned by BIL-02-ADJ-01 and the wallet refund lifecycle by BIL-05. PLM-CFG-05 answers: *for adjustment reason code X, what are the rules?*

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_PLM-CFG-05-Adjustment_Type_Catalog-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `GET /api/adjustment-reason-code-defaults/{operator}/EQUIPMENT_RETURN_AT_DISCONNECT`
- `GET /api/adjustment-reason-code-defaults/{operator}/INSTALL_FAILED_REFUND`
- `GET /api/adjustment-reason-code-defaults/{operator}/{system_event_type}`
- `GET /api/adjustment-reason-code-defaults?operatorCode=WIK`

- `GET /api/adjustment-reason-codes/{operator}/{reason_code}`
- `GET /api/adjustment-reason-codes/{operator}/{reason_code}/history`
- `GET /api/adjustment-reason-codes?operatorCode=WIK&active=true`
- `GET /api/adjustment-reason-codes?operatorCode=X&active=true`
- `GET /api/adjustment-reason-codes?operatorCode=X&adjustmentKind=WALLET_REFUND&active=true`
- `GET /api/adjustment-reason-codes?operatorCode=X&category=WAIVER&active=true`
- `PATCH /api/adjustment-reason-codes/{operator}/{reason_code}`
- `POST /api/adjustment-reason-codes`
- `POST /api/adjustment-reason-codes/{operator}/{rc}/validate-proposal`
- `POST /api/adjustment-reason-codes/{operator}/{reason_code}/validate-proposal`
- `PUT /api/adjustment-reason-code-defaults/{operator}/{system_event_type}`
- `PUT /api/adjustment-reason-codes/{operator}/{reason_code}/deactivate`
- `PUT /api/adjustment-reason-codes/{operator}/{reason_code}/reactivate`

4.2 Tables

- `adjustment_reason_code`
- `adjustment_reason_code_history`
- `adjustment_reason_code_operator_default`

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `account-balance-waived`
- `adj-approval-auto-v1`
- `adj-approval-country-head-v1`
- `adj-approval-finance-only-v1`
- `adj-approval-fraud-team-v1`
- `adj-approval-supervisor-v1`
- `adj-approval-tiered-v1`
- `adjustment-credit-applied`
- `adjustment-debit-applied`
- `deposit-refund-applied`
- `late-fee-waived`
- `promo-credit-applied`
- `sophix-superadmin`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.

- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group P:** Catalogs (PLM family) | **Version:** v1.0 | **Module:** Product Lifecycle Management — Adjustment Type Catalog

The operator-extensible catalog of **adjustment reason codes** + their associated business rules: what kind of adjustment is this (credit / debit / refund / waiver), what approval workflow applies, which GL account it posts to, what scope is permitted (full invoice / line-item / free-amount), and what direction is allowed (only credit, only debit, or both). Owned at operator scope. Referenced by BIL-02-ADJ-01 Invoice Adjustments (every adjustment carries a reason code from this catalog), by BIL-05 Wallet Refunds (refund-from-wallet operations), by OSR-01 Stock Chain (deposit refunds at disconnect), and by BIL-04 Dunning Engine (dunning waivers and write-offs).

This is **catalog-only** — PLM-CFG-05 does not propose, approve, or post adjustments. The adjustment lifecycle is owned by BIL-02-ADJ-01 and the wallet refund lifecycle by BIL-05. PLM-CFG-05 answers: *for adjustment reason code X, what are the rules?*

1. What this module is

When Finance issues a goodwill credit because a customer was double-charged for a service outage, when a CSR waives a dunning late fee for a VIP customer, when the system auto-refunds an equipment deposit at disconnect, when an admin writes off a tax assessment — every one of these events is an **adjustment**, and every adjustment must carry a **reason code**. The reason code is the bridge between the business event (why) and the financial posting (how the GL is hit, who approved it, what document gets generated).

Without a managed catalog, reason codes proliferate informally as free text across BIL-02-ADJ-01 + BIL-05 + dunning waiver UIs, with no consistency in approval rules, GL mappings, or reporting. PLM-CFG-05 fixes that: every adjustment reason code is registered here once, configured per operator with its associated rules, and all consuming modules read from this catalog.

What this module does

1. **Register adjustment reason codes** — operator-scoped catalog of stable codes (`ADJ-GOODWILL-OUTAGE`, `ADJ-EQUIPMENT-DEPOSIT-RETURN`, `ADJ-BILLING-ERROR-WRONG-PACKAGE`, `ADJ-DUNNING-WAIVER`, `ADJ-WRITE-OFF-UNCOLLECTABLE`, etc.) with display names + descriptions.
2. **Classify each reason code** — by `direction` (CREDIT_ONLY / DEBIT_ONLY / BOTH), by `category` (DISPUTE / GOODWILL / BILLING_ERROR / DEPOSIT / WAIVER / WRITE_OFF / OPERATIONAL), and by the kind of adjustment it produces (`INVOICE_ADJUSTMENT` / `WALLET_REFUND` / `LEDGER_ENTRY_ONLY`).
3. **Bind each reason code to its approval workflow** — references the Camunda BPMN process key that BIL-02-ADJ-01 launches for adjustments of this type. Different reason codes can require different approval depths: a goodwill credit ≤ 500 KES may auto-approve; an outage credit > 50,000 KES requires country-head sign-off.
4. **Bind each reason code to its GL accounts** — operator-specific GL account references (credit account, debit account) used by the downstream accounting export. PLM-CFG-05 stores the references; the export tool consumes them.
5. **Constrain permitted scopes** — some reason codes can only target a full invoice (`FULL`), some can target line items (`LINE`), some allow free-amount entry (`AMOUNT`). PLM-CFG-05 declares which scopes are valid per reason code.
6. **Track reason-code lifecycle** — reason codes can be retired (`active=false`) without losing historical references. Active set is what new adjustments can use; retired set remains queryable for audit.
7. **Provide multi-language display names** — operators serving multiple languages (YASSN French + Wolof; potentially future operators) carry primary + secondary labels.

What this module does NOT do

- **Doesn't propose, approve, or apply adjustments.** That's BIL-02-ADJ-01's lifecycle. PLM-CFG-05 is metadata only.
- **Doesn't perform refunds.** Wallet refunds are BIL-05; out-of-system refunds (M-Pesa reverse, bank wire) are out-of-V3 ops processes. PLM-CFG-05 records what kind of refund is permitted; the actual execution is elsewhere.
- **Doesn't generate credit/debit notes.** BIL-02-GEN-01 (Invoice Generation) creates the actual documents. PLM-CFG-05 supplies the metadata that drives generator behavior.
- **Doesn't own GL definitions.** Operator chart-of-accounts + GL account list is in a separate accounting-config DD (pending) or in the operator's external ERP. PLM-CFG-05 stores GL account *references* (string identifiers) without validating them.
- **Doesn't own approval workflows.** Camunda BPMN files implementing the actual approval flows live in the BIL-02-ADJ-01 + SUB-WF-FRAMEWORK-01 deployment artifacts. PLM-CFG-05 stores the BPMN process key reference; deployment + execution is elsewhere.
- **Doesn't perform amount validation.** Whether a 50,000 KES goodwill credit is unusually large is a Drools-rule concern in BIL-02-ADJ-01 (using the approval workflow's threshold logic), not a PLM-CFG-05 concern. The catalog says "this reason code requires approval workflow X"; the workflow handles the thresholds.
- **Doesn't track adjustment totals or report on them.** Reporting and analytics live in CVM, EM-04 Reporting (pending), and operator BI tooling. PLM-CFG-05 surfaces the catalog; it does not aggregate the financial impact.

Regional flexibility

Adjustment reason codes vary by operator due to:

- **Local regulatory constraints** — Senegal has specific waiver/write-off categories required by the financial regulator (BCEAO); Kenya has different reporting requirements to the Central Bank of Kenya; Uganda has its own. v1.0 ships operator-specific seed sets per regulator.
- **Operator-specific accounting practices** — Wananchi may run different GL chart-of-accounts per operator. The `credit_gl_account` + `debit_gl_account` references on each reason code are strings opaque to PLM-CFG-05; operators configure them per their accounting system.
- **Operator-specific approval policies** — some operators centralize all adjustments through Finance; others delegate small goodwill credits to franchise managers. The `approval_workflow_bpmn_key` per reason code expresses this without requiring schema change.
- **Language** — YASSN reason code display names carry French primary + optional Wolof secondary. KE/UG/TZ use English.

The schema supports all four operators without DDL change. Per-operator seed scripts populate the catalog at deployment.

Glossary

Term	Meaning
Reason code	The stable operator-scoped identifier for one kind of adjustment (e.g., `ADJ-GOODWILL-OUTAGE`). The bridge between business event and financial posting. Referenced by every adjustment in BIL-02-ADJ-01, every wallet refund in BIL-05, and the deposit-refund flow in OSR-01.
Direction	Whether the reason code can produce CREDIT_ONLY (reduces customer balance), DEBIT_ONLY (increases customer balance), or BOTH. Most goodwill / dispute reasons are CREDIT_ONLY; billing-error corrections may be BOTH; some operational reasons (late fee, reactivation) are DEBIT_ONLY.
Category	A high-level grouping of reason codes for reporting + UI organization: DISPUTE, GOODWILL, BILLING_ERROR, DEPOSIT, WAIVER, WRITE_OFF, OPERATIONAL. Each reason code is in exactly one category.
Adjustment kind	What the reason code produces operationally: `INVOICE_ADJUSTMENT` (credit/debit note against an invoice, handled by BIL-02-ADJ-01), `WALLET_REFUND` (refund from local wallet, handled by BIL-05), `LEDGER_ENTRY_ONLY` (GL posting without customer-facing document, used for internal corrections).
Permitted scope	What targets the reason code can apply to: `FULL` (entire invoice), `LINE` (specific line items), `AMOUNT` (free-form amount), or any subset. BIL-02-ADJ-01's proposal UI offers the operator the scopes permitted by the chosen reason code.
Approval workflow	The Camunda BPMN process key launched when an adjustment of this reason code is proposed. The actual workflow definition (steps, approvers, thresholds) is in the BPMN; PLM-CFG-05 carries only the reference.
GL account reference	An operator-specific string identifying the General Ledger account that the adjustment posts to. PLM-CFG-05 stores the string; downstream accounting export consumes it. Two references per reason code: `credit_gl_account` (where credits land) and `debit_gl_account` (where debits originate).

2. Tables overview + cross-DD anchoring

2.1 Tables overview

PLM-CFG-05 owns three tables:

Table	Purpose
`adjustment_reason_code`	The master catalog: code, display names, direction, category, kind, scopes, GL accounts, approval workflow, active state.
`adjustment_reason_code_history`	Append-only audit trail of changes to reason code rows (who changed what, when, why).
`adjustment_reason_code_operator_default`	Per-operator default reason code mapping for system-initiated adjustments (e.g., "deposit refund at disconnect uses `ADJ-EQUIPMENT-DEPOSIT-RETURN`").

Full DDL in §3.

2.2 Cross-DD anchoring

PLM-CFG-05 is a pure upstream catalog — every consumer reads, none writes. No FKs FROM other modules INTO PLM-CFG-05 are declared as constraints; consumers carry the `reason_code` as a TEXT field and join logically. (This intentional decoupling means a retired reason code does not cascade-block historical records.)

3. Schemas

3.1 `adjustment_reason_code`

```
CREATE TABLE adjustment_reason_code (
  operator_code              TEXT NOT NULL,
  reason_code                TEXT NOT NULL,                 -- 'ADJ-GOODWILL-OUTAGE' | 'ADJ-EQUIPMENT-DEPOSIT-RETURN' | e
  display_name_primary       TEXT NOT NULL,                 -- 'Goodwill credit — service outage'
  display_name_secondary     TEXT NULL,                     -- bilingual; YASSN French
  description                TEXT NULL,
  direction                  TEXT NOT NULL,                 -- 'CREDIT_ONLY' | 'DEBIT_ONLY' | 'BOTH'
  category                   TEXT NOT NULL,                 -- 'DISPUTE' | 'GOODWILL' | 'BILLING_ERROR' | 'DEPOSIT' | 'WA
  adjustment_kind            TEXT NOT NULL,                 -- 'INVOICE_ADJUSTMENT' | 'WALLET_REFUND' | 'LEDGER_ENTRY_ONL
  permitted_scopes           TEXT[] NOT NULL,               -- e.g., '{FULL, LINE}' or '{AMOUNT}' or '{FULL, LINE, AMOUNT
  credit_gl_account          TEXT NULL,                     -- GL account string for the credit side; operator-defined
  debit_gl_account           TEXT NULL,                     -- GL account string for the debit side; operator-defined
  approval_workflow_bpmn_key TEXT NULL,                     -- Camunda BPMN process key; NULL = zero-step (auto-approve)
  max_amount                 NUMERIC(12,2) NULL,            -- optional hard cap (in operator currency); NULL = no cap
  amount_currency            TEXT NOT NULL,                 -- 'KES' | 'UGX' | 'TZS' | 'XOF'
  requires_justification     BOOLEAN NOT NULL DEFAULT TRUE, -- whether the proposer must enter free-text justification
  triggers_customer_notice   BOOLEAN NOT NULL DEFAULT TRUE, -- whether DD_NOT-01 sends a customer notice after applicatio
  notice_template_code       TEXT NULL,                     -- if triggers_customer_notice, the NOT-01 template to use
  active                     BOOLEAN NOT NULL DEFAULT TRUE,
  effective_from             DATE NOT NULL,
  effective_to               DATE NULL,                     -- NULL = current
  notes                      TEXT NULL,
  created_at                 TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at                 TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by                 TEXT NOT NULL,
  PRIMARY KEY (operator_code, reason_code),
  CHECK (direction IN ('CREDIT_ONLY','DEBIT_ONLY','BOTH')),
  CHECK (adjustment_kind IN ('INVOICE_ADJUSTMENT','WALLET_REFUND','LEDGER_ENTRY_ONLY')),
  CHECK (array_length(permitted_scopes, 1) > 0)
);

CREATE INDEX idx_arc_operator_active ON adjustment_reason_code (operator_code, active);
```

```
CREATE INDEX idx_arc_category        ON adjustment_reason_code (operator_code, category, active);
CREATE INDEX idx_arc_kind            ON adjustment_reason_code (operator_code, adjustment_kind, active);
```

Sample data (KE WIK):

reason_code	display_name_primary	direction	category	adjustment_kind	permitted_scopes	credit_gl_account	approval_workflow_bpmn_key	max_amount (KES)	notice_template_code
ADJ-GOODWILL-OUTAGE	Goodwill credit — service outage	CREDIT_ONLY	GOODWILL	INVOICE_ADJUSTMENT	{FULL,LINE,AMOUNT}	4500-GOODWILL-CREDITS	adj-approval-tiered-v1	50000	adjustment-credit-applied
ADJ-GOODWILL-RETENTION	Goodwill credit — retention offer	CREDIT_ONLY	GOODWILL	INVOICE_ADJUSTMENT	{AMOUNT}	4500-GOODWILL-CREDITS	adj-approval-tiered-v1	20000	adjustment-credit-applied
ADJ-DISPUTE-WRONG-PACKAGE	Dispute — wrong package billed	CREDIT_ONLY	DISPUTE	INVOICE_ADJUSTMENT	{LINE}	4510-BILLING-CORRECTIONS	adj-approval-tiered-v1	NULL	adjustment-credit-applied
ADJ-DISPUTE-OVERCHARGE	Dispute — overcharged amount	CREDIT_ONLY	DISPUTE	INVOICE_ADJUSTMENT	{FULL,LINE,AMOUNT}	4510-BILLING-CORRECTIONS	NULL	adj-approval-tiered-v1	adjustment-credit-applied
ADJ-BILLING-ERROR-DEBIT	Billing error — missed charge (debit)	DEBIT_ONLY	BILLING_ERROR	INVOICE_ADJUSTMENT	{AMOUNT}	NULL	adj-approval-finance-only-v1	NULL	adjustment-debit-applied
ADJ-EQUIPMENT-DEPOSIT-RETURN	Equipment deposit refund	CREDIT_ONLY	DEPOSIT	WALLET_REFUND	{AMOUNT}	4600-DEPOSITS-PAYABLE	adj-approval-auto-v1	NULL	deposit-refund-applied
ADJ-EQUIPMENT-DEPOSIT-FORFEIT	Equipment deposit forfeiture	DEBIT_ONLY	DEPOSIT	LEDGER_ENTRY_ONLY	{AMOUNT}	4600-DEPOSITS-PAYABLE	adj-approval-finance-only-v1	NULL	NULL
ADJ-DUNNING-WAIVER-LATE-FEE	Dunning waiver — late fee	CREDIT_ONLY	WAIVER	INVOICE_ADJUSTMENT	{LINE}	4520-DUNNING-WAIVERS	adj-approval-supervisor-v1	5000	late-fee-waived
ADJ-DUNNING-WAIVER-FULL	Dunning waiver — full balance	CREDIT_ONLY	WAIVER	INVOICE_ADJUSTMENT	{FULL}	4520-DUNNING-WAIVERS	adj-approval-country-head-v1	NULL	account-balance-waived
ADJ-WRITE-OFF-UNCOLLECTABLE	Write-off — uncollectable	CREDIT_ONLY	WRITE_OFF	LEDGER_ENTRY_ONLY	{FULL}	4530-BAD-DEBT-WRITE-OFF	adj-approval-country-head-v1	NULL	NULL
ADJ-WRITE-OFF-FRAUD	Write-off — fraud	CREDIT_ONLY	WRITE_OFF	LEDGER_ENTRY_ONLY	{FULL}	4540-FRAUD-WRITE-OFF	adj-approval-fraud-team-v1	NULL	NULL
ADJ-PROMO-COMPENSATION	Promo compensation—operational	CREDIT_ONLY	OPERATIONAL	INVOICE_ADJUSTMENT	{AMOUNT}	4550-PROMO-CREDITS	adj-approval-auto-v1	10000	promo-credit-applied

Notes on the sample:

- Row 1 `ADJ-GOODWILL-OUTAGE` permits all three scopes — CSR can credit the whole invoice, individual lines, or enter a free amount. Capped at 50,000 KES (above that, BIL-02-ADJ-01 forces escalation regardless of approval workflow).
- Row 5 `ADJ-BILLING-ERROR-DEBIT` is DEBIT_ONLY — used when a charge was missed and needs to be added; rare but possible.
- Row 6 `ADJ-EQUIPMENT-DEPOSIT-RETURN` is `WALLET_REFUND` kind — when a customer returns equipment at disconnect, BIL-05 issues a refund from the customer's wallet using this reason code. `adj-approval-auto-v1` workflow auto-approves (no human step) because the qty + amount are deterministic from OSR-01 + PLM-CFG-06 historical lookup.
- Row 7 `ADJ-EQUIPMENT-DEPOSIT-FORFEIT` is `LEDGER_ENTRY_ONLY` — the customer never sees a customer-facing document; the GL is hit to move the deposit from "payable" to "operator revenue" when forfeit conditions are met.
- Row 11 `ADJ-WRITE-OFF-FRAUD` has a specialized fraud-team approval workflow (segregation of duties).

YASSN reason codes carry French primary names; e.g., `ADJ-GOODWILL-PANNE` (`display_name_primary='Avoir commercial — panne de service'`). Same shape; per-operator seed.

3.2 `adjustment_reason_code_history`

Append-only audit log of changes to `adjustment_reason_code` rows. Every UPDATE to the reason code table writes one row here, with full before/after snapshots.

```
CREATE TABLE adjustment_reason_code_history (
  history_id            TEXT PRIMARY KEY,                 -- 'arch_01HZ...'
  operator_code         TEXT NOT NULL,
  reason_code           TEXT NOT NULL,
  change_type           TEXT NOT NULL,                    -- 'CREATED' | 'UPDATED' | 'DEACTIVATED' | 'REACTIVATED'
  before_state          JSONB NULL,                       -- full row before change (NULL on CREATED)
  after_state           JSONB NOT NULL,                   -- full row after change
  fields_changed        TEXT[] NULL,                      -- e.g., '{max_amount, approval_workflow_bpmn_key}'
  changed_at            TIMESTAMP NOT NULL DEFAULT NOW(),
  changed_by            TEXT NOT NULL,
  change_reason         TEXT NULL                         -- free-text justification for the change
);

CREATE INDEX idx_arch_reason_code ON adjustment_reason_code_history (operator_code, reason_code, changed_at DESC);
CREATE INDEX idx_arch_changed_at  ON adjustment_reason_code_history (changed_at DESC);
```

Sample data:

history_id	operator_code	reason_code	change_type	fields_changed	changed_by	changed_at	change_reason
arch_01HZ_GOODWILL_CREATED	WIK	ADJ-GOODWILL-OUTAGE	CREATED	NULL	seed-script-v1.0	2026-01-15 10:00:00	Initial v1.0 seed
arch_01HZ_GOODWILL_CAP_RAISED	WIK	ADJ-GOODWILL-OUTAGE	UPDATED	{max_amount}	bo_admin_finance_03	2026-04-12 14:30:00	Raised from 25k to 50k after Q1 review; outages of major ISPs in March warranted higher single-credit ceilings
arch_01HZ_DUNNING_WAIVER_NEW_WORKFLOW	WIK	ADJ-DUNNING-WAIVER-LATE-FEE	UPDATED	{approval_workflow_bpmn_key}	bo_admin_finance_03	2026-05-01 09:00:00	Moved from finance-only to supervisor-only to reduce friction for small waivers

This history is used for compliance audits (regulators often ask "show me the change log of your adjustment policy"). v1.0 keeps it forever; future archival policy is out of scope.

3.3 `adjustment_reason_code_operator_default`

When a system-initiated adjustment fires (e.g., OSR-01 RETURN movement triggering deposit refund), the system needs to know which reason code to use. This table maps system event types to default reason codes per operator.

```
CREATE TABLE adjustment_reason_code_operator_default (
  operator_code         TEXT NOT NULL,
  system_event_type     TEXT NOT NULL,                    -- 'EQUIPMENT_RETURN_AT_DISCONNECT' | 'EQUIPMENT_FORFEIT_AT_DUN
  reason_code           TEXT NOT NULL,                    -- FK composite (operator_code, reason_code) → adjustment_reas
  notes                 TEXT NULL,
  updated_at            TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_by            TEXT NOT NULL,
  PRIMARY KEY (operator_code, system_event_type),
  FOREIGN KEY (operator_code, reason_code) REFERENCES adjustment_reason_code(operator_code, reason_code)
);
```

Sample data:

operator_code	system_event_type	reason_code	notes
WIK	EQUIPMENT_RETURN_AT_DISCONNECT	ADJ-EQUIPMENT-DEPOSIT-RETURN	OSR-01 default for return-flow deposit refund
WIK	EQUIPMENT_FORFEIT_AT_DUNNING_TERMINATION	ADJ-EQUIPMENT-DEPOSIT-FORFEIT	BIL-04 dunning final-step deposit forfeit

operator_code	system_event_type	reason_code	notes
WIK	INSTALL_FAILED_REFUND	ADJ-GOODWILL-OUTAGE	FUL-02 cancellation refunds use goodwill code by default
WIK	OUTAGE_AUTO_CREDIT_BATCH	ADJ-GOODWILL-OUTAGE	BIL-04 outage credit batch (per workshop OutageCredit batch flow)
YASSN	EQUIPMENT_RETURN_AT_DISCONNECT	ADJ-EQUIPMENT-DEPOSIT-RETURN	YASSN-specific reason code (French primary name)

When OSR-01 records a RETURN movement, the deposit-refund routine looks up `system_event_type='EQUIPMENT_RETURN_AT_DISCONNECT'` for the operator, gets back `ADJ-EQUIPMENT-DEPOSIT-RETURN`, and uses that as the reason code for the BIL-05 wallet refund. The mapping makes system-initiated adjustments operator-configurable without code change.

4. Rules

Rule	Description
R-PLM-ARC-1	Reason codes are operator-scoped and stable. The conventional prefix is `ADJ-<CATEGORY>-<SUB-DETAIL>` (e.g., `ADJ-GOODWILL-OUTAGE`, `ADJ-DUNNING-WAIVER-LATE-FEE`); operators MAY deviate. Once a reason code is used by any adjustment (BIL-02-ADJ-01 or BIL-05 has written a row referencing it), the code cannot be deleted — only deactivated.
R-PLM-ARC-2	A reason code with `active=false` is hidden from BIL-02-ADJ-01 and BIL-05 proposal UIs and rejected by their APIs if proposed. Historical adjustments referencing the deactivated reason code remain queryable.
R-PLM-ARC-3	`direction` constrains what kind of adjustment can be proposed: `CREDIT_ONLY` rejects DEBIT proposals; `DEBIT_ONLY` rejects CREDIT proposals; `BOTH` accepts either. The constraint is enforced at the BIL-02-ADJ-01 + BIL-05 API layer at proposal time.
R-PLM-ARC-4	`permitted_scopes` defines the scope subset the proposer can choose from. If a reason code permits only `{LINE}`, the proposer must select line items; the UI does not offer FULL or AMOUNT options. Enforced at the API layer.
R-PLM-ARC-5	`adjustment_kind` routes the adjustment to the right downstream module: `INVOICE_ADJUSTMENT` → BIL-02-ADJ-01 generates credit/debit note via BIL-02-GEN-01; `WALLET_REFUND` → BIL-05 issues refund from local wallet; `LEDGER_ENTRY_ONLY` → only the GL posting happens (no customer-facing document). The proposer UI / API enforces correct routing.
R-PLM-ARC-6	`max_amount` is a hard cap: BIL-02-ADJ-01 + BIL-05 reject proposals exceeding `max_amount` regardless of approval workflow outcome. This cap is independent of, and additive to, the approval workflow's own thresholds.
R-PLM-ARC-7	`approval_workflow_bpmn_key`: when an adjustment is proposed, BIL-02-ADJ-01 launches a Camunda BPMN process instance using this key. If the key is NULL, the adjustment auto-approves (zero-step). If the key is set but the corresponding BPMN is not deployed at launch time, BIL-02-ADJ-01 rejects the proposal with `APPROVAL_WORKFLOW_NOT_DEPLOYED`.
R-PLM-ARC-8	`credit_gl_account` and `debit_gl_account` are operator-defined opaque strings. PLM-CFG-05 stores them; downstream accounting export consumes them. The values can be NULL (e.g., a reason code that's pure customer-facing without GL posting); but if non-NULL, must follow the operator's chart-of-accounts convention.
R-PLM-ARC-9	Every UPDATE to `adjustment_reason_code` writes one row to `adjustment_reason_code_history` capturing the before/after snapshots and the changing user + reason. INSERT and soft-delete (active flip) also produce history rows. The audit trail is append-only and immutable.
R-PLM-ARC-10	`triggers_customer_notice = TRUE` causes DD_NOT-01 to send a customer notification (via the `notice_template_code` template) after the adjustment is applied. The notification is the customer's visibility into the adjustment. For LEDGER_ENTRY_ONLY adjustments, this is typically FALSE (the customer doesn't see internal corrections).
R-PLM-ARC-11	`requires_justification = TRUE` means the BIL-02-ADJ-01 / BIL-05 proposer must enter free-text justification on the proposal. v1.0 ships all reason codes with this TRUE; operators can set FALSE for highly-routine reason codes (e.g., auto-batch outage credits).
R-PLM-ARC-12	`adjustment_reason_code_operator_default` is consulted by system-initiated flows (OSR-01 deposit refund, BIL-04 outage credit batch, FUL-02 cancellation refund). User-initiated adjustments (CSR-proposed via BIL-02-ADJ-01 UI) ignore this table — the user picks the reason code from the operator's active catalog.

5. REST API

5.1 Catalog query

```
GET /api/adjustment-reason-codes?operatorCode=WIK&active=true HTTP/1.1
Authorization: Bearer <staff-or-service-jwt>

→ 200 OK
{
  "reasonCodes": [
    {
      "operatorCode":              "WIK",
      "reasonCode":                "ADJ-GOODWILL-OUTAGE",
      "displayNamePrimary":        "Goodwill credit — service outage",
      "direction":                 "CREDIT_ONLY",
      "category":                  "GOODWILL",
      "adjustmentKind":            "INVOICE_ADJUSTMENT",
      "permittedScopes":           ["FULL","LINE","AMOUNT"],
      "creditGlAccount":           "4500-GOODWILL-CREDITS",
      "debitGlAccount":            null,
      "approvalWorkflowBpmnKey":   "adj-approval-tiered-v1",
      "maxAmount":                 50000,
      "amountCurrency":            "KES",
      "requiresJustification":     true,
      "triggersCustomerNotice":    true,
      "noticeTemplateCode":        "adjustment-credit-applied",
      "active":                    true
    },
    ...
  ],
  "totalCount": 12
}
```

Filter parameters: `category`, `adjustmentKind`, `direction`, `active`.

```
GET /api/adjustment-reason-codes/{operator}/{reason_code}             # detail
GET /api/adjustment-reason-codes/{operator}/{reason_code}/history     # change history
```

5.2 Catalog management (BO admin)

```
POST /api/adjustment-reason-codes HTTP/1.1
Authorization: Bearer <bo-admin-jwt>
Content-Type: application/json

{
  "operatorCode":              "WIK",
  "reasonCode":                "ADJ-NEW-CATEGORY-V1",
  "displayNamePrimary":        "New adjustment kind",
  "direction":                 "CREDIT_ONLY",
  "category":                  "OPERATIONAL",
  "adjustmentKind":            "INVOICE_ADJUSTMENT",
  "permittedScopes":           ["AMOUNT"],
  "creditGlAccount":           "4555-NEW-CREDITS",
  "approvalWorkflowBpmnKey":   "adj-approval-tiered-v1",
  "amountCurrency":            "KES",
  "effectiveFrom":             "2026-06-01",
  "changeReason":              "Approved by Q2 finance committee"
}

→ 201 Created
{
  "operatorCode":  "WIK",
  "reasonCode":    "ADJ-NEW-CATEGORY-V1",
  "historyId":     "arch_01HZ_NEW_CREATED"
}

PATCH /api/adjustment-reason-codes/{operator}/{reason_code}           # update (writes history row)
PUT /api/adjustment-reason-codes/{operator}/{reason_code}/deactivate  # soft-delete
PUT /api/adjustment-reason-codes/{operator}/{reason_code}/reactivate  # un-deactivate
```

All mutations require `<operator>-finance-admin` role; cross-operator changes require `sophix-superadmin`.

5.3 Default mapping

```
GET /api/adjustment-reason-code-defaults?operatorCode=WIK HTTP/1.1

→ 200 OK
```

```
{
  "defaults": [
    { "systemEventType": "EQUIPMENT_RETURN_AT_DISCONNECT", "reasonCode": "ADJ-EQUIPMENT-DEPOSIT-RETURN" },
    { "systemEventType": "EQUIPMENT_FORFEIT_AT_DUNNING_TERMINATION", "reasonCode": "ADJ-EQUIPMENT-DEPOSIT-FORFEIT" },
    { "systemEventType": "OUTAGE_AUTO_CREDIT_BATCH", "reasonCode": "ADJ-GOODWILL-OUTAGE" },
    ...
  ]
}
```

```
GET /api/adjustment-reason-code-defaults/{operator}/{system_event_type}   # single lookup, used by OSR-01 etc.
PUT /api/adjustment-reason-code-defaults/{operator}/{system_event_type}   # operator admin sets default
```

The single-lookup endpoint is the hot path for system-initiated adjustments: OSR-01 RETURN movement → `GET .../WIK/EQUIPMENT_RETURN_AT_DISCONNECT` → receives `ADJ-EQUIPMENT-DEPOSIT-RETURN` → passes to BIL-05 wallet refund API.

5.4 Validation helper

For consuming modules that want to pre-validate a proposed adjustment before invoking the full BIL-02-ADJ-01 proposal API:

```
POST /api/adjustment-reason-codes/{operator}/{reason_code}/validate-proposal HTTP/1.1
Authorization: Bearer <service-jwt>
Content-Type: application/json

{
  "direction":  "CREDIT",
  "scope":      "LINE",
  "amount":     12500,
  "currency":   "KES"
}

→ 200 OK
{
  "valid":      true,
  "reasonCode": "ADJ-GOODWILL-OUTAGE",
  "willTriggerApproval":     true,
  "approvalWorkflowBpmnKey": "adj-approval-tiered-v1"
}
```

Or:

```
{
  "valid":   false,
  "reason":  "DIRECTION_NOT_PERMITTED",
  "details": "Reason code ADJ-EQUIPMENT-DEPOSIT-RETURN is CREDIT_ONLY; cannot propose DEBIT"
}
```

6. Kafka events

PLM-CFG-05 publishes on `sophix.plm.arc.*`.

Event type	Topic	Emitted when	Consumers
`AdjustmentReasonCodeCreated`	`sophix.plm.arc.created`	New reason code	BIL-02-ADJ-01 cache refresh, BIL-05 cache refresh
`AdjustmentReasonCodeUpdated`	`sophix.plm.arc.updated`	Reason code fields changed	All consumers — cache invalidation
`AdjustmentReasonCodeDeactivated`	`sophix.plm.arc.deactivated`	`active=true → false`	All consumers — remove from active picker
`AdjustmentReasonCodeReactivated`	`sophix.plm.arc.reactivated`	`active=false → true`	All consumers — add back to picker
`AdjustmentReasonCodeDefaultChanged`	`sophix.plm.arc.default-changed`	Operator default mapping changed	OSR-01, BIL-04, FUL-02 — refresh default lookups

6.1 Sample event — `AdjustmentReasonCodeUpdated`

```
{
  "eventType":   "AdjustmentReasonCodeUpdated",
  "aggregateId": "WIK::ADJ-GOODWILL-OUTAGE",
  "timestamp":   "2026-04-12T14:30:00.000Z",
  "payload": {
    "operatorCode":    "WIK",
    "reasonCode":      "ADJ-GOODWILL-OUTAGE",
    "fieldsChanged":   ["maxAmount"],
    "before": {
      "maxAmount":     25000,
```

```
        "amountCurrency": "KES"
      },
      "after": {
        "maxAmount":     50000,
        "amountCurrency": "KES"
      },
      "historyId":       "arch_01HZ_GOODWILL_CAP_RAISED",
      "changedBy":       "bo_admin_finance_03",
      "changeReason":    "Raised from 25k to 50k after Q1 review; outages of major ISPs in March warranted higher single-cre
      "effectiveImmediately": true
    }
  }
```

Consumers refresh their reason-code caches; in-flight proposals continue with the rules in effect at proposal time (rule snapshot is captured by BIL-02-ADJ-01 at proposal commit).

7. Cross-DD coordination

7.1 Upstream callers

Caller	Endpoint	When
BIL-02-ADJ-01 (Invoice Adjustments)	`GET /api/adjustment-reason-codes?operatorCode=X&active=true`	Adjustment proposal UI — populate the reason code picker
BIL-02-ADJ-01	`POST /api/adjustment-reason-codes/{operator}/{rc}/validate-proposal`	Pre-validate the proposal before persisting
BIL-05 (Wallet Refunds)	`GET /api/adjustment-reason-codes?operatorCode=X&adjustmentKind=WALLET_REFUND&active=true`	Refund proposal UI
BIL-04 (Dunning Engine)	`GET /api/adjustment-reason-codes?operatorCode=X&category=WAIVER&active=true`	Waiver proposal at L2 escalation
OSR-01 (Stock Chain)	`GET /api/adjustment-reason-code-defaults/{operator}/EQUIPMENT_RETURN_AT_DISCONNECT`	RETURN movement → triggers deposit refund using the operator default
FUL-02 (Cancellation flow)	`GET /api/adjustment-reason-code-defaults/{operator}/INSTALL_FAILED_REFUND`	Order cancellation refund
BO admin UI	various	Catalog management + history view

7.2 Downstream consumers (Kafka)

Consumer	Events subscribed	Action
BIL-02-ADJ-01	`AdjustmentReasonCodeUpdated`, `AdjustmentReasonCodeDeactivated`, `AdjustmentReasonCodeReactivated`	Refresh reason-code cache; flag in-flight proposals if their reason code was deactivated
BIL-05	Same	Refresh cache
BIL-04	Same	Refresh cache; in-flight dunning waivers using a deactivated code are flagged for ops review
OSR-01	`AdjustmentReasonCodeDefaultChanged`	Refresh default lookup cache for `EQUIPMENT_RETURN_AT_DISCONNECT` + `EQUIPMENT_FORFEIT_AT_DUNNING_TERMINATION`
BO admin ops dashboard	All `sophix.plm.arc.*`	Display the catalog + change history
Compliance / audit tooling	All `sophix.plm.arc.*`	Regulator-required change log

7.3 Sibling modules

Module	Direction	Contract
BIL-02-ADJ-01	BIL-02-ADJ-01 → PLM-CFG-05 (read)	Every adjustment carries a `reason_code` from this catalog. Approval workflow key drives Camunda launch.
BIL-05 (Wallet Refunds)	BIL-05 → PLM-CFG-05 (read)	Refunds carry a reason code; PLM-CFG-05 says which reason codes are WALLET_REFUND kind.
BIL-04 (Dunning Engine)	BIL-04 → PLM-CFG-05 (read)	Dunning waivers and write-offs reference reason codes from category WAIVER / WRITE_OFF.

Module	Direction	Contract
OSR-01 (Stock Chain)	OSR-01 → PLM-CFG-05 (read)	OSR-01 RETURN movement triggers deposit refund using the operator default reason code from `adjustment_reason_code_operator_default`. The cross-DD reference in OSR-01 §8.2 is to this DD (correction of earlier reference to PLM-CFG-08).
DD_NOT-01 (Notifications)	NOT-01 ← PLM-CFG-05 (informational)	When `triggers_customer_notice=TRUE` and an adjustment is applied, BIL-02-ADJ-01 / BIL-05 invokes NOT-01 with the `notice_template_code` from the reason code. PLM-CFG-05 itself does not call NOT-01.
PLM-CFG-08 (Bundle Catalog)	No relationship	Despite the PLM-CFG-NN numbering proximity, PLM-CFG-08 is the Bundle Catalog (sets of packages); it does not interact with the adjustment reason code catalog.
Accounting export (pending)	Accounting ← PLM-CFG-05 (read)	The GL account references on each reason code feed into the accounting export job that produces operator chart-of-accounts entries.

7.4 Dependencies (build order)

1. **BIL-02-ADJ-01** (Invoice Adjustments) — already shipped at v1.x; provides the consumer side. ADJ-01 v1.x referenced "operator's reason code catalog" as an external concept; PLM-CFG-05 v1.0 IS that catalog.
2. **Per-operator reason code seed** — per-operator seed scripts populate ~10–15 reason codes per operator at deployment (WIK seed shown in §3.1 sample data; YASSN, WUG, WTZ each get their own French / English / Swahili-aware seed). Seed includes operator-default mappings.
3. **Approval workflow BPMN deployment** — Camunda BPMN files implementing the named approval workflows (`adj-approval-tiered-v1`, `adj-approval-finance-only-v1`, `adj-approval-supervisor-v1`, `adj-approval-country-head-v1`, `adj-approval-auto-v1`, `adj-approval-fraud-team-v1`) must be deployed before reason codes referencing them go live. BPMN deployment is owned by BIL-02-ADJ-01 + SUB-WF-FRAMEWORK-01 infrastructure.
4. **GL account string registry** — operator chart-of-accounts strings must be registered with the operator's accounting system before they're used in reason code GL fields. PLM-CFG-05 itself doesn't validate the strings.
5. **OSR-01 + BIL-04 + BIL-05** — read this catalog; coexistence with v1.0 OSR-01 means OSR-01's earlier cross-reference to PLM-CFG-08 for deposit refund is corrected to point at PLM-CFG-05 in OSR-01 §8.3 (correction noted in this DD's §7.3).

8. Workshop sources

No dedicated workshop deep-dive for Adjustment Type Catalog. The functional area is implicit in:

- **Discounts workshop** (deep-dive 11) — adjustments and discounts are operationally related (both reduce customer-payable amounts); the workshop discusses goodwill credits and dispute resolution.
- **Payments + Collections workshop** (deep-dive 04) — dunning waivers, write-offs, deposit refunds — all use reason codes from this catalog.
- **Invoices workshop** (deep-dive 10) — credit notes against invoices reference reason codes.

V3 design intent: **decouple reason codes from the adjustment lifecycle modules** (BIL-02-ADJ-01, BIL-05, BIL-04) so the catalog can evolve operationally (new codes, new approval workflows, new GL mappings) without code change in the consuming modules. The TZ codebase's current handling of adjustment reason codes is unverified pending cartography; placeholder: "TBD pending codebase review."

Out of v1.0 scope:

- **Approval threshold rules expressed in the catalog row** — v1.0 puts threshold logic in the approval workflow BPMN, not in the reason code row. A future enhancement may add per-amount-tier metadata directly on the row.
- **GL account validation** — v1.0 stores GL strings opaquely; future enhancement may add validation against an operator chart-of-accounts table.
- **Operator-level approval policy versioning** — when an operator changes their approval workflow for a reason code mid-year, in-flight proposals continue with the older workflow snapshot. Full versioning (multiple approval workflow rows

per reason code, each effective-dated) is out of v1.0.

- **Cross-reason-code analytics** — total credits issued per category per month, fraud-write-off trend lines, etc. — out; CVM and EM-04 Reporting (pending) own this.
- **Multi-currency same-reason-code pricing** — v1.0 supports one `amount_currency` per reason code per operator. Multi-currency is out.

End of DD_PLM-CFG-05-Adjustment_Type_Catalog-v1.0.